Career Pilot– AI-Based Personalized Student Career Advisor

PROJECT REPORT FOR THE COURSE
AD23731 FOUNDATIONS OF AGENTIC AI

Submitted by

SAILESH RANGARAJ. (Reg. No. 231501141)
UMESH SARATHY SK. (Reg. No. 231501177)
VIGNESH BALA R. (Reg. No. 231501180)



DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI, 602105

RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the mini-project work done by **Sailesh Rangaraj (231501141), Umesh Sarathy S K (231501177) Vignesh Bala R (231501180),** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Practical Examination held on: ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

S.No	TOPIC	PAGE No
1.	**Introduction**	**5**
2.	**Problem Statement and Business Case**	**7**
3.	**User and Stakeholder Analysis**	**9**
4.	**Proposed Agentic AI Solution**	**10**
5.	**Dataset / Knowledge Sources**	**12**
6.	**Tools and Technologies Used**	**13**
7.	**Agent Architecture**	**15**
8.	**Agent Workflow and Orchestration**	**18**
9.	**Agentic RAG Implementation**	**19**
10.	**System Implementation**	**22**
11.	**Experimental Evaluation and Results**	**25**
12.	**Comparison, Innovation and Scalability**	**27**
13.	**Conclusion and Future Work**	**29**

ABSTRACT

This project presents a **Career Pilot** an Agentic AI-based personalized Student Career Advisor designed to help students translate their academic profile, skills and career interests into actionable career recommendations. Unlike a conventional question-answering chatbot, the proposed system uses an agent that can reason over a task, select tools, inspect tool outputs, generate a plan, critique its own generated application content, and pause for human approval before consequential actions.

The system is implemented as a Flask application around a handwritten ReAct-style agent loop. The agent can analyze student information, search a job database, identify skill gaps and prepare job application material. A Reflection phase reduces the risk of fabricated or exaggerated claims: a second LLM call reviews the generated resume/cover letter/answers against the candidate's actual profile and either approves the draft or corrects unsupported claims. An Agentic RAG layer allows the agent to decide when additional career or skill-roadmap knowledge should be retrieved.

The evaluation design compares No RAG, Basic RAG and Agentic RAG and tests the same student profiles across multiple LLM configurations. Evaluation focuses on relevance, skill-gap accuracy, learning-plan quality, grounding, consistency, retrieval efficiency and response time.

1.INTRODUCTION

1.1Background and Motivation

Choosing an appropriate career path is an important decision for students, particularly for students who are approaching graduation and entering the job market. A student may have knowledge of several programming languages, databases, analytical tools, academic subjects, projects, internships, or certifications, but may not know how these skills correspond to actual job roles. The difficulty is not only finding a suitable job but also understanding the skills required for the role, identifying missing skills, deciding what to learn next, and preparing an appropriate application.

Traditional career guidance generally depends on manually searching job portals, reading job descriptions, consulting career websites, following learning roadmaps, and discussing opportunities with mentors or placement coordinators. These activities require considerable time and the information obtained from different sources may not be connected. A student may therefore receive generic recommendations that do not sufficiently consider the student's current skills and career goal.

The proposed project, **Career Pilot– AI-Based Personalized Student Career Advisor**, addresses this problem by combining Large Language Model reasoning with tools, structured student information, job information, document retrieval, reflection, and human approval. The system is designed as an agentic workflow rather than a simple question-answering chatbot.

The agent receives information about the student and the intended career direction. It can analyze the profile, search available jobs, identify skill gaps, retrieve relevant career and skill-roadmap information, generate a personalized learning plan, prepare application content, verify the generated content, and finally request human approval before an application action is performed.

The project therefore treats career advising as a **multi-step decision and execution workflow** rather than a single text-generation task.

1.2Need for Agentic AI

A conventional LLM can generate a career recommendation from a prompt, but a career-advising workflow requires several actions to be performed in sequence. The system must determine what information is already available, decide which tool should be used, inspect the output of that tool, determine whether additional information is required, and then continue the task.

For example, consider a student interested in becoming a Data Analyst. The system may first examine the student's profile and identify Python, SQL and basic statistics as existing

skills. It may then search available jobs and find that many relevant positions additionally require Power BI, data visualization and advanced SQL. The agent can identify these as gaps and decide whether external knowledge is required to create a meaningful learning roadmap.

This is where the agentic approach becomes useful. Instead of always following exactly the same sequence, the agent can decide what action is appropriate based on the current state of the task.

The major agentic characteristics implemented in the project are:

- **Planning and reasoning:** understanding the student's request and determining the next useful operation.
- **Tool usage:** invoking job search and skill-analysis functionality when required.
- **Observation:** examining the results returned by tools.
- **Adaptive retrieval:** deciding whether additional career or learning knowledge is necessary.
- **Reflection:** reviewing generated application material for unsupported claims.
- **State handling:** maintaining application-related status in the system.
- **Human-in-the-Loop:** requiring approval before consequential application actions.

The project therefore demonstrates how an LLM can act as an orchestrator of a workflow instead of functioning only as a text generator.

1.3 Project Objectives

The primary objective is to develop a working Agentic AI system that provides personalized career assistance to students.

The specific objectives are:

1. To design a single-agent architecture for student career advising.
2. To implement a handwritten ReAct-style agent loop.
3. To analyze student profiles and identify relevant skills.
4. To search a structured job database for suitable opportunities.
5. To compare student skills with job requirements.
6. To identify skill gaps that affect career readiness.
7. To introduce a curated career and skill-roadmap knowledge base.
8. To implement Agentic RAG in which the agent decides when additional knowledge is required.
9. To generate personalized learning plans based on identified gaps.
10. To generate application-related content such as resumes, cover letters or answers.
11. To implement Reflection and self-correction for generated application content.
12. To prevent unsupported candidate claims from being treated as verified facts.
13. To implement a Draft → Review → Approve → Apply workflow.
14. To retain human control over consequential application actions.
15. To compare No RAG, Basic RAG and Agentic RAG.
16. To evaluate the system using consistent student profiles and multiple LLM configuration

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

The central problem addressed by the project is the lack of a unified system that connects a student's current profile, career interests, available job opportunities, skill gaps and learning requirements.

A student may know what career they are interested in but may not know:

- Which jobs match their current profile.
- Which skills are missing.
- Which missing skills should be learned first.
- What resources or roadmaps should be followed.
- Whether generated application content accurately represents their actual experience.
- When external knowledge is required to improve the recommendation.

A simple chatbot can answer questions about careers, but it generally does not provide a complete workflow involving profile analysis, job search, gap identification, retrieval, planning, verification and approval.

Career AI attempts to connect these stages into a single agentic process.

2.2 Real-World Context

The system is particularly relevant to college students preparing for internships, placements and entry-level employment. It can also support placement cells, academic mentors and career-support teams.

For example, a placement cell may have a large number of students seeking guidance simultaneously. Providing individual manual guidance for every student can be time-consuming. The proposed system can perform first-level analysis and generate structured recommendations while allowing faculty or students to remain involved in important decisions.

The system is not intended to replace human career mentors. Instead, it provides an intelligent assistance layer that can organize information and perform repetitive analysis.

2.3 Business Value

Challenge	Conventional approach	Proposed agentic approach
Career planning	Manual search across sources	Agent coordinates a multi-step workflow
Skill gaps	Generic recommendations	Compare profile with job requirements
Knowledge use	Static or always-on retrieval	Agent decides whether/what to retrieve
Applications	Single-pass generation	Generation followed by Reflection
External action	Manual workflow	Agent drafts; human approves

2.4 Success Criteria

The project considers the following indicators when evaluating the system:

2.4.1 Relevance of recommended jobs.
2.4.2 Accuracy of identified skill gaps.
2.4.3 Relevance of generated learning plans.
2.4.4 Grounding of recommendations in retrieved knowledge.
2.4.5 Reduction of unsupported application claims through Reflection.
2.4.6 Appropriate use of retrieval.
2.4.7 Consistency across repeated executions.
2.4.8 Response latency.
2.4.9 Successful enforcement of human approval before application actions.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

User	Needs	System Support
Student	Career direction, skill gaps, learning plan, application help	Profile analysis, jobs, roadmap, draft applications
Placement / Career Cell	Scalable first-level advising	Structured recommendations and workflow
Faculty / Mentor	Evidence for advising discussions	Skill-gap and roadmap summaries
Administrator	Manage jobs and knowledge sources	Database and knowledge-base management

3.2 Placement and Career Cell

Placement cells can use the system to support students at scale. Instead of manually performing the initial comparison between every student's profile and available job opportunities, the system can generate an initial analysis.
The placement team can then focus on higher-level mentoring and decision-making.

3.3 Faculty and Mentors

Faculty members or mentors can use the generated skill-gap analysis and learning plans as supporting material during career discussions.
The system does not remove the mentor from the process. Human judgment remains important, particularly when the recommendation has significant consequences for a student's career.

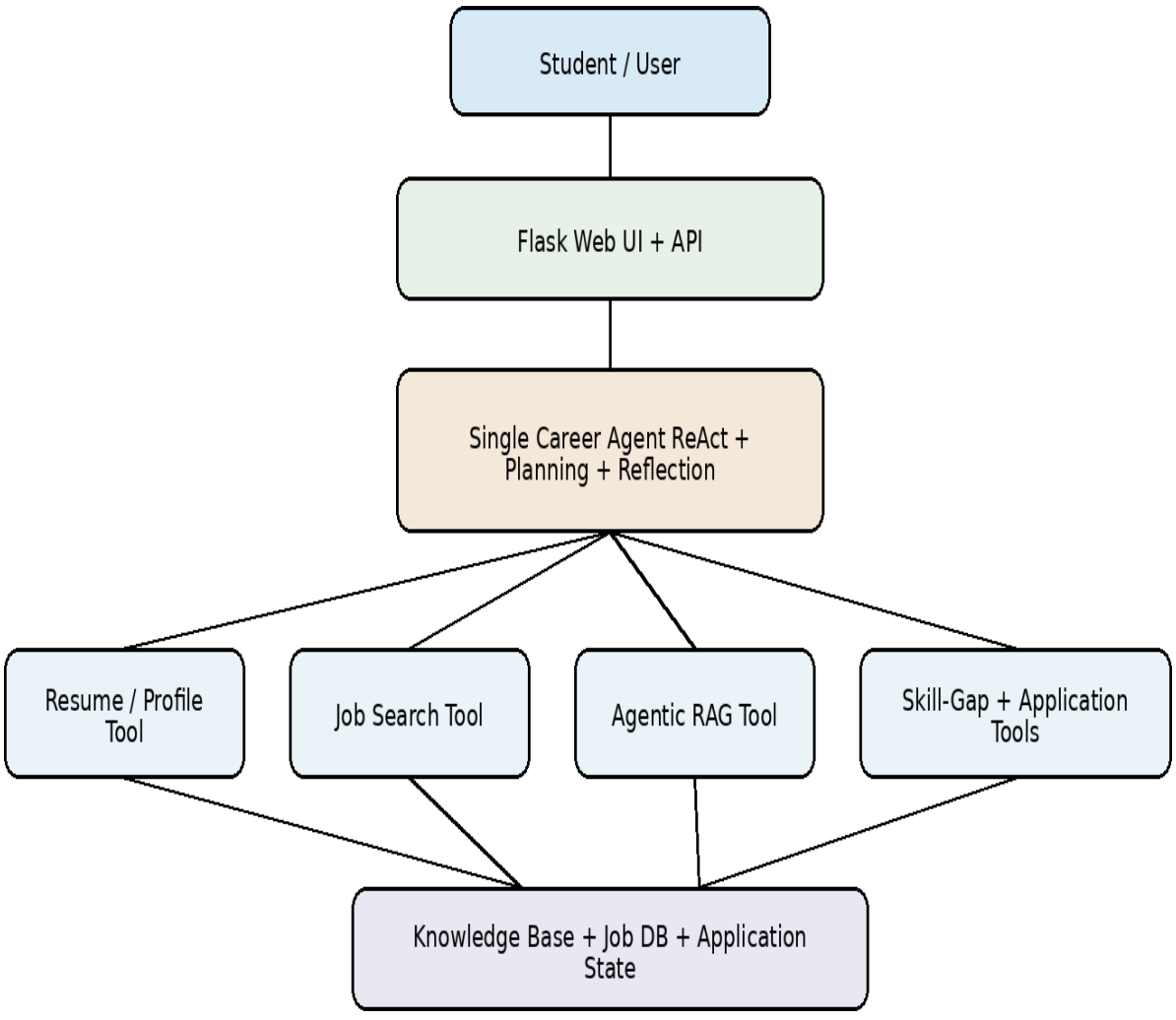
3.4 System Administrator

The administrator is responsible for maintaining structured job information and the career knowledge base. The quality of the system therefore depends partly on the quality and relevance of the information supplied to the system.

4 . PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

Career AI is implemented as a Flask-based web application containing a single tool-using agent.
The agent acts as the central coordinator. Instead of implementing every stage as an independent fixed pipeline, the agent determines which operation should be performed based on the current task.
The general workflow is:
Student Profile → Agent Analysis → Tool Selection → Job Search → Skill Gap Analysis → Agentic Retrieval → Learning/Application Generation → Reflection → Human Approval
The system therefore combines structured data processing and LLM reasoning.



4.2 Key Features

Resume/profile understanding. Tool-driven job discovery.
Skill-gap identification.

Agentic retrieval from career and skill-roadmap documents.

Personalized learning-plan generation.
Reflection/self-critique for application content.

Draft → Approve → Apply Human-in-the-Loop workflow.

Persistent application status in the database.

4.3 Major System Features

The system provides the following major features:

Profile Analysis

The agent analyzes the student's academic and technical profile and extracts useful information for further processing.

Job Discovery

The agent can invoke the job-search functionality to identify relevant opportunities from the structured job database.

Skill Gap Analysis

The system compares existing student skills with the requirements of the selected role or job.

Agentic RAG

The agent determines whether the available information is sufficient or whether additional knowledge should be retrieved from the career knowledge base.

Personalized Learning Plan

The system generates a learning roadmap based on the identified gaps.

Reflection

Generated application content is reviewed by another LLM call before it is considered verified.

Human Approval

The final application action requires explicit approval.

5 DATASET / KNOWLEDGE SOURCES

5.1 Structured Data

The application uses structured student/profile information and a job database. Job records contain role titles, skills or requirements and application-related fields. The search_jobs function operates over this structured data. It is useful tool-based retrieval but is distinct from document-based RAG.

5.2 Knowledge Base for Agentic RAG

A small curated knowledge base is introduced to satisfy the document-retrieval requirement. It contains career guides and skill roadmaps such as data-analyst roadmaps, Python developer roadmaps, SQL guidance, Power BI guidance, machine-learning roadmaps and interview-preparation material.

Knowledge source	Example content	Use
Career roadmap	Role → skills → progression	Career direction and priorities
Skill roadmap	Beginner → intermediate → advanced	Learning sequence
Tool-specific guide	SQL / Power BI / Python guidance	Fill identified gaps
Interview guide	Preparation areas	Interview readiness

5.3 Data Governance

Use small curated documents for predictable
demonstration. Keep title, source and topic metadata.
Chunk documents into meaningful retrieval
units. Preserve source identifiers.
Do not treat retrieved text as permission to invent candidate facts.

6 .TOOLS AND TECHNOLOGIES USED

Technology / Component	Role
Python	Core implementation language
Flask	Web application and API layer
Groq-served LLMs	Generation, reasoning and Reflection
Database	Student/application state and job data
Vector retrieval layer	Knowledge-base retrieval
HTML / CSS / JavaScript	User interface
Handwritten ReAct loop	Agent orchestration
Git / GitHub	Version control and repository

6.1 Python:

Python is used as the primary implementation language. It handles the agent loop, tool execution, database interaction and Flask application logic.

6.2 Flask:

Flask provides the web application and API layer.
It connects the user interface with the underlying agent functionality.

6.3 Groq-Served LLMs:

Groq-served LLMs are used for generation, reasoning and Reflection.
The evaluation can use multiple model configurations so that the behavior of the system is not evaluated using only one model.

6.4 Database:

The database stores structured information such as student/application state and job information. The application state allows the system to maintain the status of an application workflow.

6.5 Vector Retrieval Layer:

The vector retrieval component supports the document knowledge base used by Agentic RAG. Documents are divided into meaningful chunks, converted into embeddings and indexed so that relevant knowledge can be retrieved.

6.6 Handwritten ReAct Loop:

The project intentionally uses a handwritten agent loop rather than relying on LangChain or LangGraph. This makes the internal reasoning and tool orchestration visible and easier to explain in an academic project.

7.AGENT ARCHITECTURE

7.1 Agent Components

Component	Responsibility
Planner / Reasoner	Understand goal and decide next action
Tool selector	Choose appropriate tool
Job search tool	Retrieve matching structured jobs
Skill-gap tool	Compare profile and requirements
RAG controller	Decide if knowledge retrieval is needed
Generator	Produce roadmap/application content
Reflection reviewer	Check and correct unsupported output
State layer	Persist application status
Human approval	Authorize final action

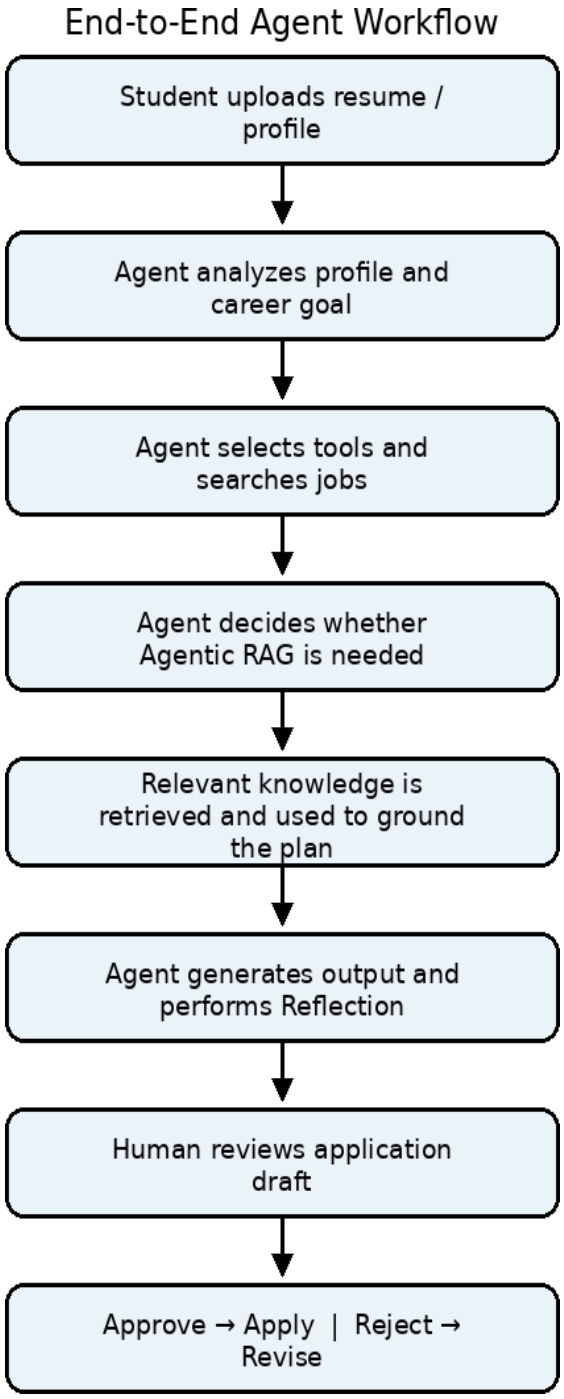
7.2 ReAct-Style Reasoning

The system follows the basic ReAct idea of alternating between reasoning and action. A simplified representation is:

Reason → Act → Observe → Reason → Act → Observe → Final Response For example:

1. The agent receives a request for a suitable career plan.
2. It determines that profile information is required.
3. It analyzes the profile.
4. It determines that job information is useful.
5. It invokes the job-search tool.
6. It observes job requirements.
7. It identifies skill gaps.
8. It decides whether additional career knowledge is necessary.
9. It retrieves relevant roadmap information if required.
10. It generates the learning plan.
11. It returns the final recommendation.

This demonstrates that the agent is not simply generating a response from one prompt.



7.3 Reflection / Self-Correction

Reflection is applied particularly to application-related
generation. The first LLM call generates the draft.
A second LLM call acts as a reviewer.
The reviewer checks whether the generated material is supported by:

7.3.1 Student skills.
7.3.2 Education.
7.3.3 Projects.
7.3.4 Experience.

If the draft contains unsupported or exaggerated claims, the reviewer identifies and corrects them.
For example, if a student profile contains Python and SQL but no Machine Learning experience, the system should not generate a statement claiming professional Machine Learning experience.
The verification response includes information such as:

7.3.5 verified
7.3.6 review_notes

If the reviewer succeeds without finding an issue, the output can be marked verified. If a problem is detected, the content can be corrected.
If the reviewer call fails because of a network or API problem, the system does not falsely claim that verification was successful.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

The workflow is organized around agent decisions rather than a fixed sequence. The agent can stop when enough information is available or invoke additional tools when observations reveal missing information.

8.2 Task Decomposition

Understand request and target
career/application. Analyze profile.
Search jobs when opportunity information is
needed. Compare requirements with profile.
Identify gaps and decide whether domain knowledge is
needed. Perform Agentic RAG when required.
Generate a learning plan or application
draft. Run Reflection and revise
unsupported claims. Present result to the
student.
Wait for human approval before applying.

8.3 Agent Coordination

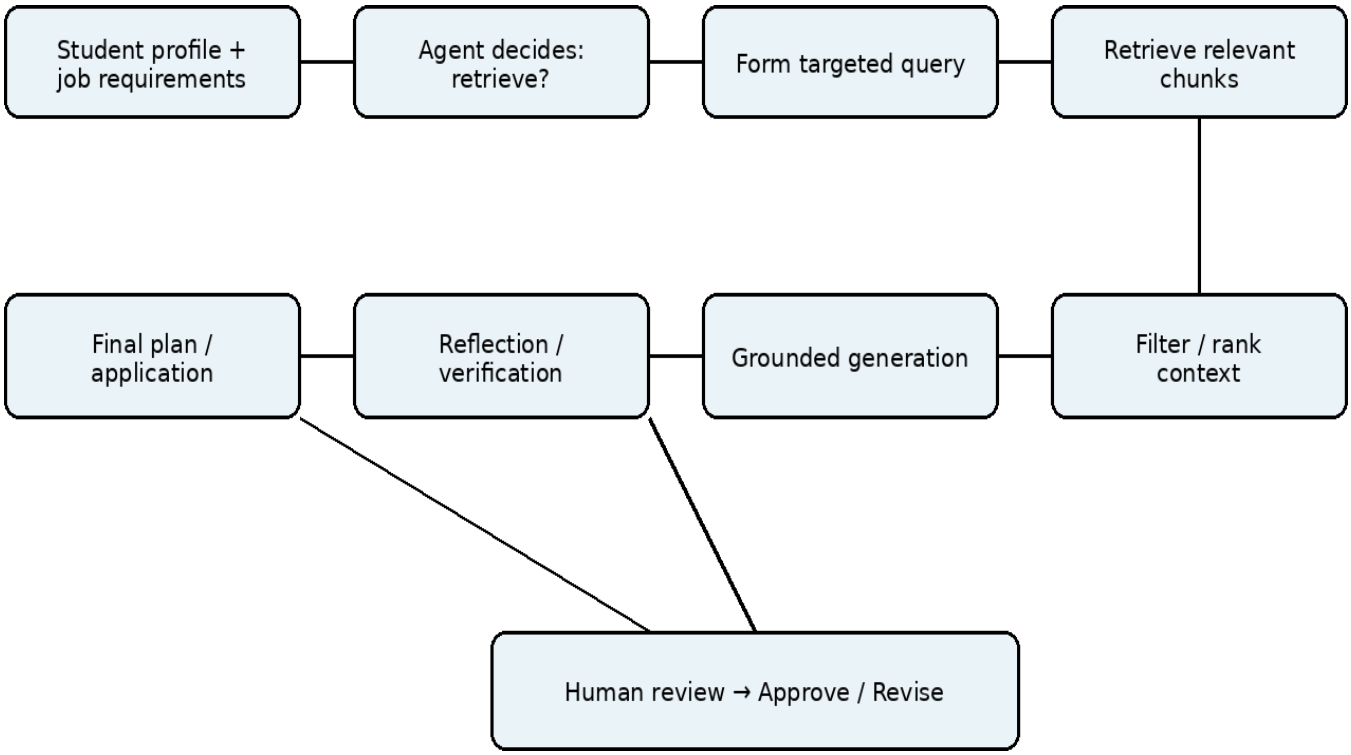
The single agent coordinates all tools. Coordination is achieved through structured inputs and observations rather than independent agent-to-agent communication.

8.4 Orchestration Implementation

The LLM selects the next useful operation, while the Python application layer validates and executes tool calls. This separation prevents the model from directly performing unrestricted database operations.

9.AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture



The key distinction is control. In Basic RAG, retrieval is performed by a fixed pipeline. In Agentic RAG, the agent first decides whether additional knowledge is required. If yes, it forms a targeted query, retrieves candidate chunks, filters or ranks them and uses the selected context to ground generation.

9.2 No RAG

In the No RAG configuration, the system does not retrieve external career documents. The model operates using the information supplied directly in the prompt and structured system data. This provides the baseline against which the retrieval approaches can be evaluated.

9.3 Basic RAG

In the Basic RAG configuration, relevant documents are retrieved using a fixed retrieval process. The retrieval step occurs without requiring the agent to make an explicit decision about whether retrieval is necessary. This represents the conventional RAG baseline.

9.4 Agentic RAG

In Agentic RAG, the agent first considers whether additional knowledge is needed. If retrieval is required, the agent creates a targeted retrieval query.

The retrieval component then:

1. Searches the knowledge base.
2. Retrieves candidate chunks.
3. Ranks or filters relevant information.
4. Returns the selected context.
5. Provides source information to the generation stage.

The agent then uses the retrieved information to produce the final recommendation.

9.5 Example of Agentic Retrieval

Suppose a student has:

9.5.1 Python

9.5.2 SQL

9.5.3 Basic statistics

and is interested in Data Analytics.

The agent searches jobs and identifies Power BI and data visualization as common requirements.

At this point, the agent may decide that additional knowledge is needed to construct a structured learning sequence.

It can query the knowledge base for:

"Data Analyst roadmap for Power BI and data visualization"

The retrieved knowledge can then help the agent construct a sequence such as:

1. Data visualization fundamentals.
2. Power BI basics.
3. Data transformation.
4. Dashboard development.
5. Practical project.
6. Interview preparation.

The exact roadmap is generated from the retrieved knowledge and the student's identified gaps.

9.6 Retrieval Quality

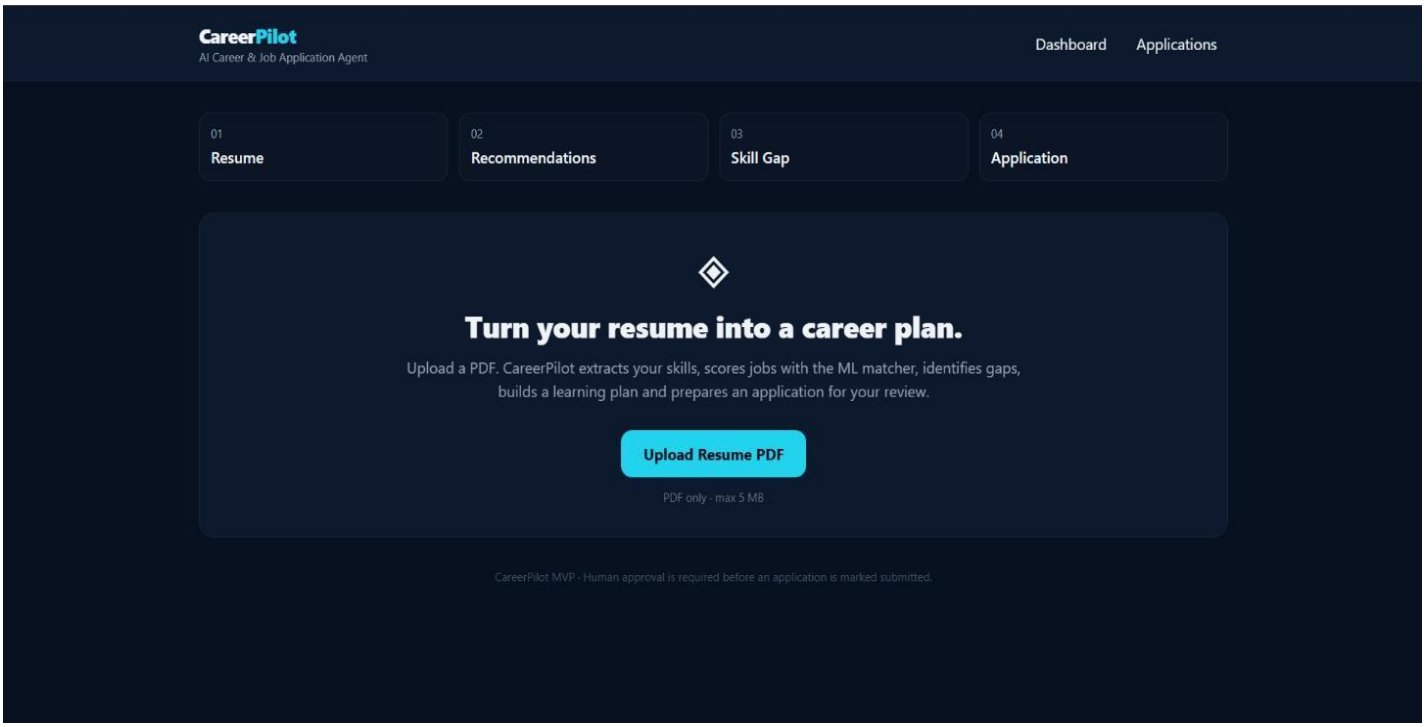
Agentic RAG is evaluated using retrieval-related measurements such as:

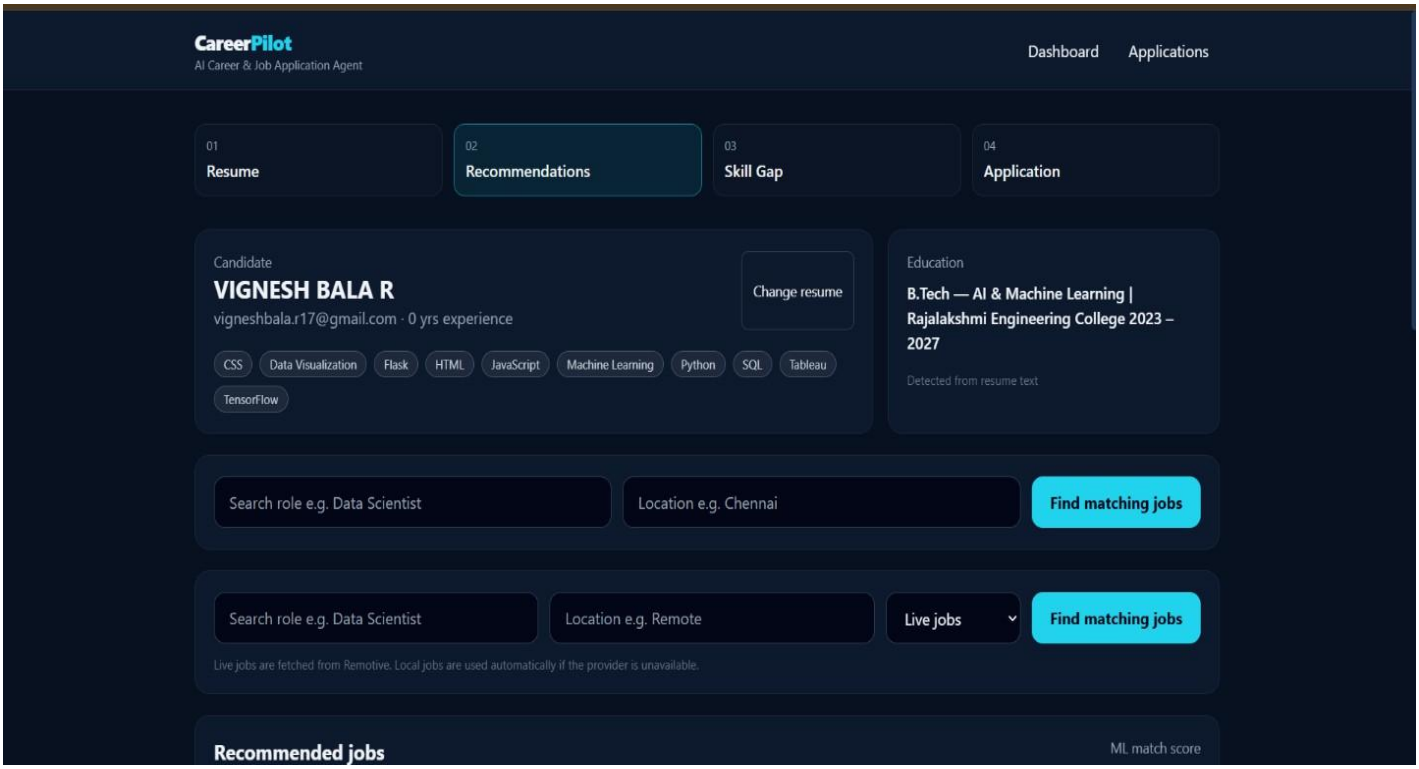
- 9.6.1 Retrieval relevance.
- 9.6.2 Context precision.
- 9.6.3 Grounding.
- 9.6.4 Number of retrieval calls.
- 9.6.5 Selective retrieval accuracy.

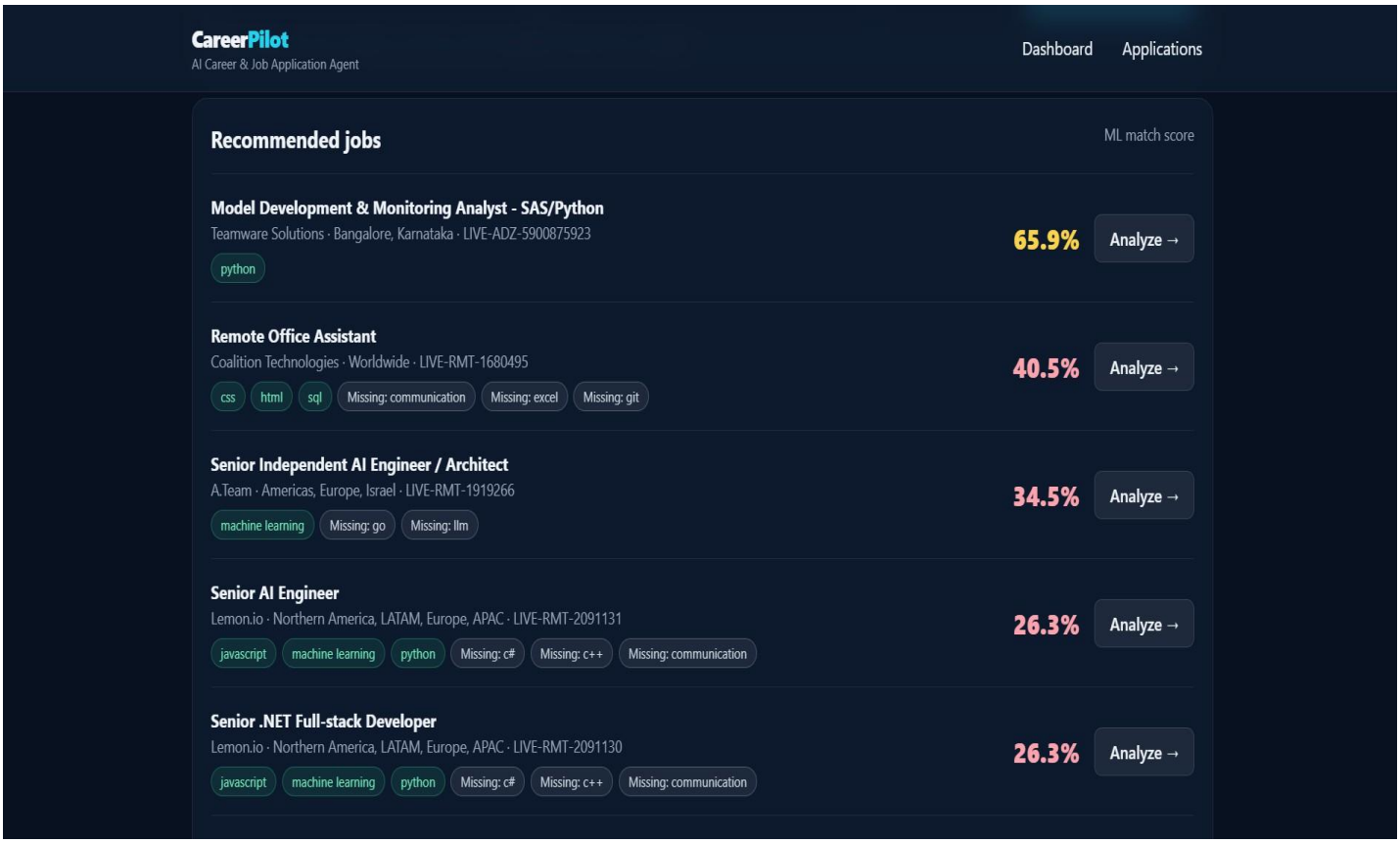
These measurements help determine not only whether the system retrieves information, but whether it retrieves information appropriately.

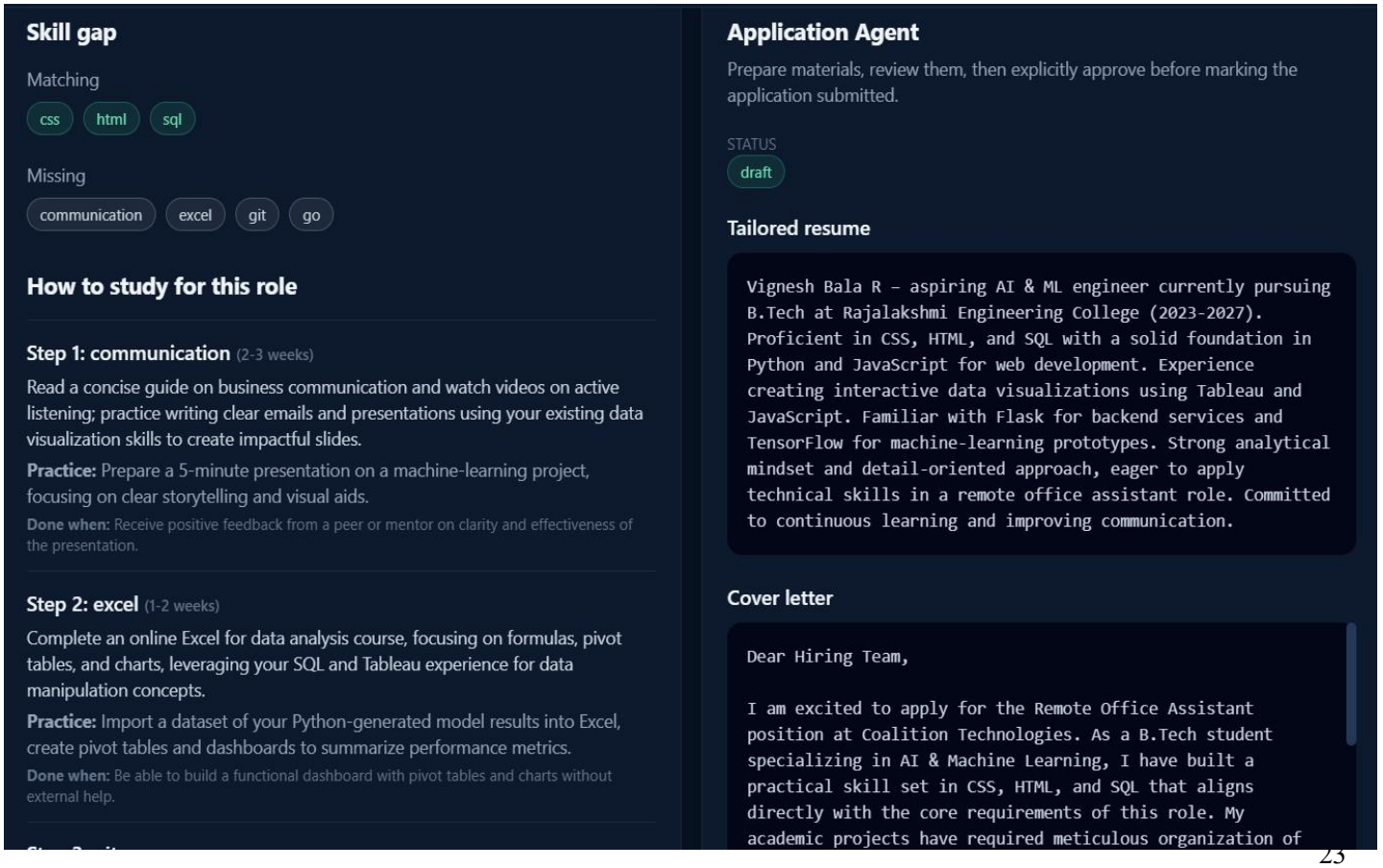
10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype






CareerPilot
AI Career & Job Application Agent
Dashboard
Applications
Recommended jobs
ML match score
Model Development & Monitoring Analyst - SAS/Python
Teamware Solutions · Bangalore, Karnataka · LIVE-ADZ-5900875923
python
65.9%
Analyze →
Remote Office Assistant
Coalition Technologies · Worldwide · LIVE-RMT-1680495
css
html
sql
Missing: communication
Missing: excel
Missing: git
40.5%
Analyze →
Senior Independent AI Engineer / Architect
A.Team · Americas, Europe, Israel · LIVE-RMT-1919266
machine learning
Missing: go
Missing: llm
34.5%
Analyze →
Senior AI Engineer
Lemon.io · Northern America, LATAM, Europe, APAC · LIVE-RMT-2091131
javascript
machine learning
python
Missing: c#
Missing: c++
Missing: communication
26.3%
Analyze →
Senior .NET Full-stack Developer
Lemon.io · Northern America, LATAM, Europe, APAC · LIVE-RMT-2091130
javascript
machine learning
python
Missing: c#
Missing: c++
Missing: communication
26.3%
Analyze →


Skill gap
Matching
css
html
sql
Missing
communication
excel
git
go
How to study for this role
Step 1: communication (2-3 weeks)
Read a concise guide on business communication and watch videos on active listening; practice writing clear emails and presentations using your existing data visualization skills to create impactful slides.
Practice: Prepare a 5-minute presentation on a machine-learning project, focusing on clear storytelling and visual aids.
Done when: Receive positive feedback from a peer or mentor on clarity and effectiveness of the presentation.
Step 2: excel (1-2 weeks)
Complete an online Excel for data analysis course, focusing on formulas, pivot tables, and charts, leveraging your SQL and Tableau experience for data manipulation concepts.
Practice: Import a dataset of your Python-generated model results into Excel, create pivot tables and dashboards to summarize performance metrics.
Done when: Be able to build a functional dashboard with pivot tables and charts without external help.
Application Agent
Prepare materials, review them, then explicitly approve before marking the application submitted.
STATUS
draft
Tailored resume
Vignesh Bala R – aspiring AI & ML engineer currently pursuing B.Tech at Rajalakshmi Engineering College (2023-2027). Proficient in CSS, HTML, and SQL with a solid foundation in Python and JavaScript for web development. Experience creating interactive data visualizations using Tableau and JavaScript. Familiar with Flask for backend services and TensorFlow for machine-learning prototypes. Strong analytical mindset and detail-oriented approach, eager to apply technical skills in a remote office assistant role. Committed to continuous learning and improving communication.
Cover letter
Dear Hiring Team,
I am excited to apply for the Remote Office Assistant position at Coalition Technologies. As a B.Tech student specializing in AI & Machine Learning, I have built a practical skill set in CSS, HTML, and SQL that aligns directly with the core requirements of this role. My academic projects have required meticulous organization of

10.2 Application Preparation and Reflection

The application-preparation endpoint was extended so that generation is followed by a second reviewer call. The response exposes verification status and review notes, making the Reflection behavior visible in the API and user interface.

10.3 Functional Prototype Screenshots

Profile upload screen.

Agent analysis and identified

skills. Job recommendations and

skill gaps.

Agentic RAG decision and retrieved knowledge.

Personalized learning roadmap.

Application draft with verification

status. Human approval screen.

Application status after approval.

Insert final application screenshots in this section. Recommended layout: two screenshots per page with figure numbers, short captions and a one-paragraph explanation.

11 .EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

Metric	Definition
Job relevance	Appropriateness of recommended jobs for profile
Skill-gap accuracy	Correspondence between identified gaps and requirements
Plan relevance	Whether roadmap addresses identified gaps
Grounding	Whether RAG-supported claims are supported by retrieved text
Consistency	Similarity across repeated runs
Verification effectiveness	Unsupported claims detected/corrected
Latency	Time from request to final response
Retrieval efficiency	Retrieval actions and useful retrieval ratio

11.2 Testing Methodology

Create a fixed set of synthetic or anonymized test resumes covering different career directions. Run every profile through No RAG, Basic RAG and Agentic RAG with the same prompts and job data. Repeat selected tests for consistency and run the same test set through two or more LLM configurations.

11.3 Reflection Evaluation

Use controlled tests. First, inject an unsupported skill into a generated draft—for example, a candidate with Python and SQL but no listed Machine Learning experience. The reviewer should flag and remove or rewrite the unsupported claim. Second, simulate a reviewer-call failure and verify that the API returns the original draft with verified=false rather than claiming successful verification.

11.4 Success-Metric Achievement

The prototype should demonstrate the complete agentic loop, observable Agentic RAG decisions, Reflection-based correction and human approval before application. Quantitative thresholds can be finalized after pilot measurements and then applied consistently.

12 .COMPARISON, INNOVATION AND SCALABILITY

12.1 Comparison with Baselines

Dimension	No RAG	Basic RAG	Agentic RAG
External knowledge	LLM knowledge only	Retrieved every time	Retrieved when agent decides
Adaptivity	Low	Moderate	High
Retrieval overhead	None	Potentially high	Task-dependent
Grounding	Limited	Improved	Improved with selective retrieval
Agent decision-making	Tool use	Fixed retrieval	Retrieval is an agent action

12.2 Advantages

Combines structured tools with LLM reasoning.
Separates candidate facts from external career knowledge. Uses Reflection to reduce unsupported application claims. Retains human control over consequential actions.
Enables a direct three-way retrieval experiment.
Keeps orchestration transparent through handwritten code.

12.2 Project Innovation

The main contribution of the project is the combination of several agent capabilities into one student-facing workflow. These include:

- Tool selection.
- ReAct-style orchestration.
- Structured job retrieval.
- Skill-gap analysis.
- Agentic document retrieval.
- Personalized planning.
- Reflection.
- Human approval.

The project therefore demonstrates how different agent capabilities can operate together rather than presenting them as isolated features.

12.4 Scalability

Expand the knowledge base.
Scale the vector index and retrieval
service. Add permitted live job APIs.
Use asynchronous execution for expensive evaluations.
Add monitoring for retrieval quality, latency and model
errors. Support multiple isolated student profiles.

12.5 Real-World Applicability

The architecture can support college placement cells, academic advising portals and student-support systems. Production deployment would additionally require privacy controls, authentication, source governance, audit logging and secure external integrations.

13 .CONCLUSION AND FUTURE WORK

13.1 Conclusion

The Career AI project demonstrates the application of Agentic AI to personalized student career advising.

The system moves beyond simple question answering by coordinating several stages of a career-planning workflow. The agent can analyze a student's profile, select tools, search opportunities, identify skill gaps, retrieve additional knowledge when necessary, generate personalized learning recommendations, prepare application content and perform Reflection before presenting important outputs to the user.

The use of Agentic RAG is particularly important because it demonstrates that retrieval itself can become an agent decision. The system is not restricted to either always retrieving or never retrieving information. Instead, the agent can determine whether additional career knowledge is useful for the current task.

The Reflection mechanism adds another layer of reliability by checking generated application content against the student's actual profile. This reduces the possibility of unsupported or exaggerated candidate claims being carried into application material.

The Human-in-the-Loop workflow further ensures that the agent does not independently perform the final consequential application action. The student remains responsible for approving the final application.

The three-way comparison between No RAG, Basic RAG and Agentic RAG provides a clear experimental framework for studying the effect of adaptive retrieval.

Overall, the project demonstrates how reasoning, tool use, retrieval, reflection, state handling and human approval can be combined into a practical Agentic AI application.

13.2 Future Work

13.1.1 Future development can extend the existing system without changing its fundamental architecture.

13.1.2 The knowledge base can be expanded with institution-specific curricula and verified learning resources.

13.1.3 Longer-term student memory can be introduced so that completed courses, projects and previously identified skill gaps can be considered in future advising sessions.

13.1.4 Live job-market integrations can provide fresher job information.

13.1.5 Hybrid retrieval can combine lexical and vector-based search.

13.1.6 Reranking can improve retrieval quality when the knowledge base becomes larger.

13.1.7 Automated evaluation judges can supplement human evaluation.

13.1.8 A multi-agent architecture can also be investigated in the future, but only if separate agents provide measurable benefits compared with the current single-agent design.

13.1.9 Finally, a production deployment would require stronger privacy controls, authentication, audit logging, secure external integrations and monitoring.